Newtonian Mechanics Revisited: Nuances, Operational Meaning, and VD Clarity

1 What we were actually trying to do

We were not trying to decide what is *true*. We were trying to describe a *decision procedure*—a human-executable protocol for building and using a theory. In the Valid Dictionary (VD) perspective, the role of a "law" is not to proclaim metaphysical truth, but to *lock* certain terms together so that:

- we can track which concepts are being introduced and where,
- we can audit which inferences rely on which commitments,
- we can surface where interpretation/recognition is doing the real work.

A key methodological stance is: *interpretability is assumed*. Just as we do not define multiplication when we write $F = ma$, we also do not attempt to define every cognitive operation that a human uses to execute the theory.

2 Newton's original framing vs modern framing

A central nuance is that Newton's *historical* second law is not written as "acceleration equals force divided by mass" in the *Principia*. Newton states (in words) that the *alteration of motion* is proportional to the impressed force, and occurs in the direction of that force. Modern $F = ma$ emerges after identifying "motion" with momentum and then specializing (e.g. constant mass) so that $dp/dt = ma$.

This matters for VD because it exposes that "the bridge term" is partly a choice of representation:

- Newton's natural bridge is *force* and *change of motion*.
- Modern teaching often uses *acceleration* as the bridge, because calculus is now standard.

VD makes this choice explicit: one can build Newtonian mechanics using either bridge, and the structure of the dictionary forces us to notice what must be defined before what.

3 Newton I is clearer when you name the hidden concept

A recurring confusion (for modern students) is that Newton I can read like a slogan unless one already knows what it is *for*. The crucial hidden concept is the calibration object:

> "*free motion*" is what defines *inertial frames*, and inertial frames are what give content to "constant velocity."

To make this explicit, we introduced an intermediate, human-recognizable state:

$$\texttt{uniform_motion_on_interval}(I) \quad \text{meaning:} \quad \forall t_1, t_2 \in I,\ \mathbf{v}(t_1) = \mathbf{v}(t_2). \tag{1}$$

and a local version

$$\texttt{uniform_motion_at_time}(t_0) \quad \text{meaning:} \quad \exists \varepsilon > 0\ \ \forall t \in (t_0 - \varepsilon, t_0 + \varepsilon),\ \mathbf{v}(t) = \mathbf{v}(t_0). \tag{2}$$

This was not about adding mathematics; it was about placing the correct intuition inside the structure. Then Newton I becomes a clean relational statement linking three ideas:

$$\texttt{free_particle} \longleftrightarrow \texttt{inertial_frame} \longleftrightarrow \texttt{uniform_motion}.$$

Importantly, we avoided writing Newton I in terms of acceleration, because we wanted "uniform motion" to carry the conceptual load.

4 Newton II as a first-degree law (modern bridge)

Once kinematics is set up, Newton II can be written as a first-degree law using acceleration as the bridge:

$$\boxed{\mathbf{F}_{\text{net}}(p,t) = m(p)\,\mathbf{a}(p,t)} \tag{3}$$

Here, VD clarity depends on what we call the new terms:

- `net_force` is an aggregate object (it is not yet "one thing"; it will later be explained as a sum).
- `inertial_mass` is the coefficient linking net force to acceleration.

A subtle conceptual improvement is to *separate* Newton II (the link $F_{\text{net}} = ma$) from the *force aggregation rule* (how F_{net} is built from contributions).

5 Force aggregation is not the same as force attribution

We discovered that the truly hard part of mechanics is not the algebra; it is the semantic step:

> Who is interacting with what, and therefore which force-contributions should be included?

This is why force bookkeeping must be separated into at least two layers:

1. **Bookkeeping layer:** define that net force is the sum of *acting forces* in an *interaction set*.
2. **Attribution layer:** decide which *acting forces* are present (the recognition step).

This distinction explains why "collision-looking" trajectories are not sufficient evidence of interaction. Two particles can approach and later separate with no interaction (e.g. they pass through each other in a toy model); the geometry alone does not reveal the causal attribution.

5.1 A key test: momentum conservation does not imply interaction

We tested a common inference mistake: "the total momentum of two particles is conserved, therefore they interacted." This is false. Conservation of total momentum of a subsystem means that the net external impulse on that subsystem is zero; it does not uniquely identify whether there were internal interactions, mediators, or even no interaction at all.

6 Why spring-in-the-middle is epistemically cleaner

Consider two particles moving toward each other but actually interacting with a spring placed between them. Their trajectories can mimic a "bounce" that might be misread as particle–particle interaction. VD exposes what matters:

- If we recognize the spring, then we can attribute *acting forces* on each particle due to the spring.
- The fact that the particles reverse direction is not itself proof of mutual interaction; it is evidence of some force attribution, which could be mediated.

In other words, the spring example demonstrates that interaction is a modeling/recognition claim, not something forced by kinematics alone.

7 Directedness, reaction, and why Newton III is tricky

We ran into a deep nuance: the mechanism that produces "equal and opposite" forces depends on how one encodes force contributions. If one stores *force values* directly, one can enforce Newton III by an explicit pair-commit rule (insert $+\mathbf{F}$ into one set and $-\mathbf{F}$ into the other). But one can also store *force rules* whose evaluation depends on which particle is the target; then sign flips may appear as a byproduct of evaluation conventions.

We decided we wanted a representation closer to historical mechanics:

> There is one canonical force on the object being acted upon; the other side is the reaction force defined as the negative of the canonical force.

This pushes Newton III away from "coincidence of coding" and toward a rule about closure of our bookkeeping under reactions.

8 Acting objects: separating interaction-capacity from particles

A major clarity gain came from introducing a new kind of primitive (conceptually):

> "Acting objects" are mathematical objects that sit *on top of* particles and carry interaction-capacity (spring, wall contact, friction interface, field source, etc.).

This resolved two problems that are usually implicit in textbook prose:

1. **Existence vs activation:** a spring is still a spring even when not stretched. If "spring" is treated only as a force law, it seems to exist only when force is nonzero. Acting objects let us represent presence independently of activity.
2. **Reverse system description:** we often recognize "there is a spring / a floor / friction" before we specify all particles and parameters. Acting objects allow system specification in that natural order.

In the VD algorithm, acting objects eventually become the mechanism that generates acting forces and specifies activation conditions. But we also recognized that determining interaction is inherently complex and often irreducible: the recognition step (contact detection, regime selection, boundary choice, "is this a spring here?") is where most hidden modeling judgment lives.

9 Newton's broader "core" text: why the philosophy sections matter

Newton's *Scholium* on absolute vs relative time/space/motion is not a replacement for the laws; it is conceptual scaffolding. It clarifies what Newton thinks "true motion" refers to and how it might be distinguished from mere appearances. Whether one agrees with Newton's absolute structure or not, the key VD lesson is that:

> there is always a layer of semantic commitment about what counts as the underlying structure vs what is directly observable.

In VD terms, such scaffolding is exactly the kind of meaning-level content that otherwise stays implicit.

10 Takeaway: where the real complexity lives

We ended with a sharper diagnosis:

> The difficulty in connecting Newtonian concepts is not mainly the equations; it is the innate complexity of determining interactions (recognition, attribution, and system boundaries).

VD does not eliminate that complexity; it localizes it. Once the recognition inputs are declared (which acting forces/objects are active), the rest becomes a mechanical pipeline:

$$\text{recognized interactions} \;\to\; \text{acting force set} \;\to\; \mathbf{F}_{\text{net}} \;\to\; \mathbf{a} \;\to\; \text{updated motion.}$$

This is the "gold standard of clarity" claim in its most defensible form: not that VD makes physics true, but that it makes the commitments and the human steps auditable.

11 Current Dictionary Log (Finalised Entries So Far)

11.1 Non-law kinematics scaffold

E1 `time` := "A real parameter $t \in \mathbb{R}$ used to order events; differences Δt are durations measured by a clock."

E2 `reference_frame` := "A convention for assigning spatial coordinates and a time coordinate t to events: an origin, axes, and a clock."

E3 `position` := "In a reference_frame, an object's location at time t represented by a vector $\mathbf{r}(t) \in \mathbb{R}^3$."

E4 `displacement` := "$\Delta\mathbf{r} = \mathbf{r}(t_2) - \mathbf{r}(t_1)$."

E5 `trajectory` := "The time-ordered curve $\{\mathbf{r}(t) : t \in I\}$ for some interval $I \subset \mathbb{R}$."

E6 `velocity` := "$\mathbf{v}(t) = d\mathbf{r}/dt$."

E7 `speed` := "$v(t) = \|\mathbf{v}(t)\|$."

E8 `acceleration` := "$\mathbf{a}(t) = d\mathbf{v}/dt = d^2\mathbf{r}/dt^2$."

E9 `path_length` := "From t_1 to t_2: $s = \int_{t_1}^{t_2} \|\mathbf{v}(t)\|\, dt$."

E10 `point_particle` := "An idealized object whose size and internal structure are neglected; its motion in a reference_frame is fully represented by a trajectory $\mathbf{r}(t)$."

E11 `straight_line` := "A set $\mathbf{r}(\lambda) = \mathbf{r}_0 + \lambda\mathbf{u}$ with fixed $\mathbf{r}_0$, nonzero direction $\mathbf{u}$, and $\lambda \in \mathbb{R}$."

11.2 Uniform motion intermediates (non-law)

E12 `uniform_motion_on_interval(I)` := "A point_particle is in uniform_motion_on_interval(I) if its velocity is the same at every time in I; i.e. $\forall t_1, t_2 \in I,\ \mathbf{v}(t_1) = \mathbf{v}(t_2)$."

E13 `uniform_motion_at_time(t_0)` := "A point_particle is in uniform_motion_at_time(t_0) if $\exists \varepsilon > 0$ such that $\forall t \in (t_0 - \varepsilon, t_0 + \varepsilon)$, $\mathbf{v}(t) = \mathbf{v}(t_0)$."

11.3 Newton I (triplet: uniform motion as bridge; no acceleration)

E14 `uniform_motion_on_interval(I)` := "The state of motion exhibited by a free_particle when described in an inertial_frame over the interval I."

E15 `inertial_frame` := "A reference_frame in which every free_particle exhibits uniform_motion_at_time(t) (equivalently: uniform_motion_on_interval(I) on any interval of free motion)."

E16 `free_particle` := "A point_particle that, when described in an inertial_frame, exhibits uniform_motion_at_time(t) (equivalently: uniform_motion_on_interval(I)) whenever treated as free."

11.4 Newton II (triplet: acceleration as bridge)

E17 `acceleration` := "For a point_particle described in an inertial_frame, acceleration equals net_force divided by inertial_mass: $\mathbf{a} = \mathbf{F}_{\texttt{net}}/\texttt{inertial_mass}$ (equivalently: $\mathbf{F}_{\texttt{net}} = \texttt{inertial_mass} \cdot \mathbf{a}$)."

E18 `net_force` := "The vector quantity satisfying `net_force` = `inertial_mass`·`acceleration` for the point_particle (in an inertial_frame)."

E19 `inertial_mass` := "The positive scalar coefficient m such that `net_force` = m·`acceleration` for the point_particle (in an inertial_frame)."

11.5 Force bookkeeping (triplet: acting forces and interaction set)

E20 `net_force(p,t)` := "For any particle p and time t, net_force(p, t) is the vector-sum of all acting_force elements in interacting_forces_set(p, t) (equivalently: $\texttt{net_force}(p,t) = \sum_{F \in \texttt{interacting_forces_set}(p,t)} F$)."

E21 `interacting_forces_set(p,t)` := "The set of acting_force vectors that the modeler judges to be acting on particle p at time t (i.e. the force-contributions currently included for p at t)."

E22 `acting_force` := "A vector force-contribution F that appears as an element of interacting_forces_set(p, t) for some particle p and time t."

11.6 Meta-typing (triplet: raw classes and recognition-based instantiation)

E23 `raw_class` := "An abstract object-type."

E24 `raw_class` := "An abstract object-type specified by class_specific_features(C); an object x is treated as instance_of(x, C) when it is recognized as possessing the class_specific_features(C)."

E25 `class_specific_features(C)` := "The set/list of features that specify raw_class C; these features are whatever a modeler must recognize x as having in order to treat x as instance_of(x, C)."

E26 `instance_of(x,C)` := "x is treated as an instance of raw_class C when x is recognized as possessing class_specific_features(C)."

E26 `Acting force.` := "*Acting force* is a force–contribution that is either a *canonical_force(acting_object)* or a *reaction_force(acting_object)*."

E28 `Canonical force.` := "A *canonical_force(acting_object)* is an acting force designated as canonical; the corresponding *reaction_force(acting_object)* is the acting force defined as the negative of it."

E29 `Reaction force.` := "A *reaction_force(acting_object)* is an acting force defined as the negative of *canonical_force(acting_object)*."

E30 `Acting object.` := An *acting_object* is an object equipped with both an *activation_condition(acting_obje* and a corresponding *canonical_force(acting_object)*.

E31 `Activation condition.` := The *activation_condition(acting_object)* is the condition under which the *canonical_force(acting_object)* is *active* (i.e. present as a canonical acting-force contribution).

E32 `Canonical force (of an acting object).` := The *canonical_force(acting_object)* is the canonical acting-force contribution associated with an *acting_object*, and it is *active* exactly when *activation_condition(acting_object)* holds.